Low-Level System Design (LLD) Handbook

1. Introduction to Low-Level Design

What is Low-Level Design (LLD)?

Low-Level Design is the detailed design phase where we break down the High-Level Design into classes, objects, methods, interfaces and their relationships. It focuses on how each component will be implemented in code.

> ★ LLD converts the HLD architecture into a blueprint that developers can directly start coding.

Goals of LLD

- ★ Clean Code
- ★ Reusability
- ★ Extensibility
- ★ Maintainability
- ★ Testability
- ★ Scalability



HLD vs LLD

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Focus	Overall architecture	Detailed design
Components	Services, Modules	Classes, Objects, Methods
Design Level	System level	Component level
Abstraction	High-level	Low-level
Output	Architecture Diagrams	Class Diagrams, UML
Answers	What components?	How to implement?

Why is LLD Important?

- Helps in writing clean and maintainable code.
- Improves reusability and extensibility.
- Makes the system scalable and testable.
- Reduces complexity by proper object design.
- Ensures good separation of concerns.

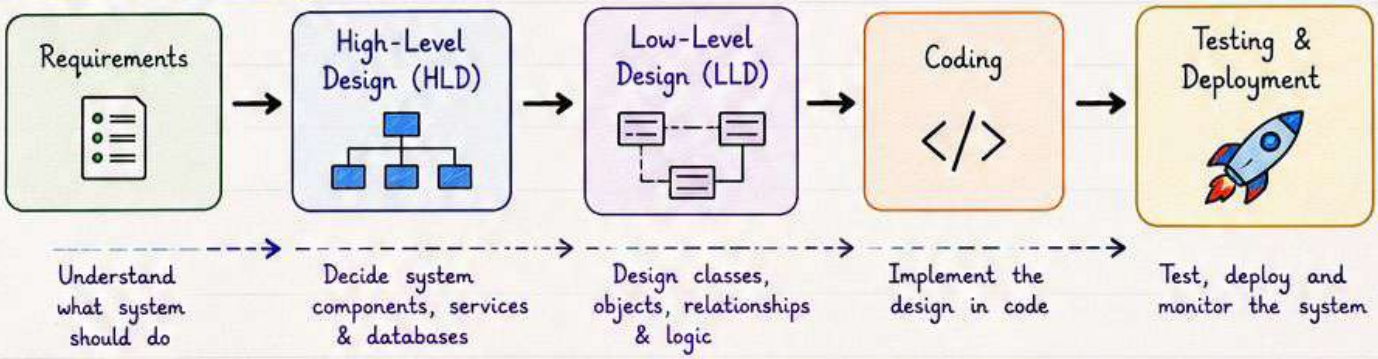
When is LLD Used?

- After HLD is finalized.
- Before actual coding begins.
- During SDE-2 and above level interviews.
- During refactoring or enhancing existing systems.

Key Elements of LLD

- ✓ Classes & Objects
- ✓ Attributes (Fields)
- ✓ Methods (Functions)
- ✓ Interfaces & Abstract Classes
- ✓ Relationships between classes
- ✓ Design Patterns
- ✓ UML Diagrams

LLD in the Big Picture



Low-Level System Design (LLD) Handbook

2. SOLID Principles

SOLID principles help us to write clean, maintainable and flexible object-oriented code. They reduce coupling and help our system to be easy to extend and test.

⭐ Why SOLID?

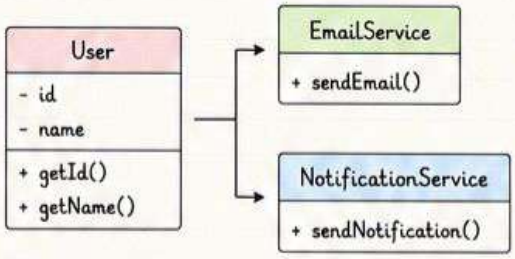
- ✓ Makes code easy to understand.
- ✓ Reduces tight coupling between components.
- ✓ Easier to maintain and extend.
- ✓ Improves testability of the system.
- ✓ Helps in building scalable applications.

1. Single Responsibility Principle (SRP)

A class should have only one reason to change, meaning it should have only one job.

Example:

- ✗ Wrong : User class handling user data, email, and notification.
- ✓ Right : Separate classes for User, EmailService, NotificationService.



Benefit

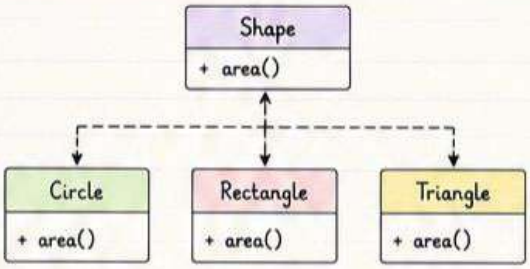
Changes in email or notification logic won't affect User class.

2. Open/Closed Principle (OCP)

Software entities (classes, modules, functions) should be open for extension but closed for modification.

Example:

- ✗ Wrong : Modifying existing code to add new features.
- ✓ Right : Use inheritance or interfaces to extend behavior.



Benefit

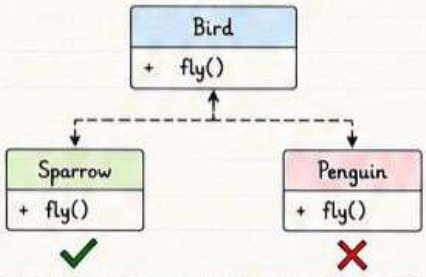
We can add new shapes without changing existing code.

3. Liskov Substitution Principle (LSP)

Objects of a superclass should be replaceable with objects of its subclass without breaking the application.

Example:

- ✗ Wrong : A child class changing the behavior of the parent class.
- ✓ Right : Child class should follow the contract of parent class.



Benefit

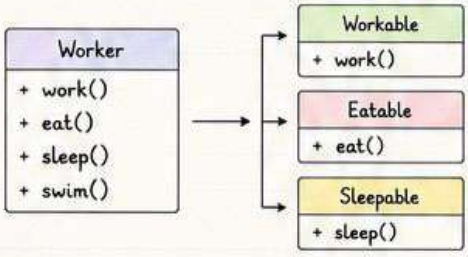
Any Bird object can be used wherever Bird is expected.

4. Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they do not use.

Example:

- ✗ Wrong : One big interface with all methods.
- ✓ Right : Create small, specific interfaces.



Benefit

Clients depend only on what they need.

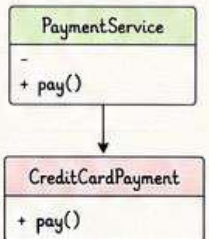
5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

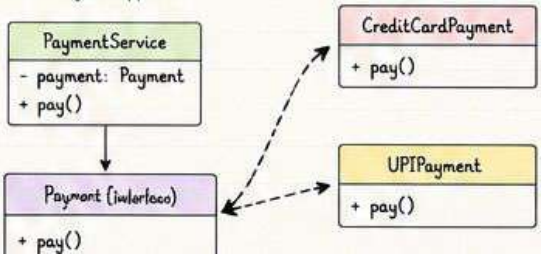
Example:

- ✗ Wrong : Class directly depends on a concrete class.
- ✓ Right : Depend on interfaces/abstractions.

✗ Wrong Approach



✓ Right Approach



Low-Level System Design (LLD) Handbook

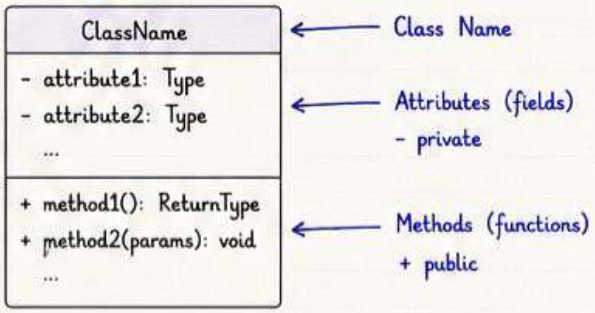
3. UML Class Diagrams

UML (Unified Modeling Language) helps us visualize the structure of our system. Class Diagram is the most commonly used diagram in LLD.

⭐ Why use Class Diagrams?

- Helps in identifying classes and their relationships.
- Acts as a blueprint before writing code.
- Improves communication in the team.
- Reduces rework and design errors.

Class Structure



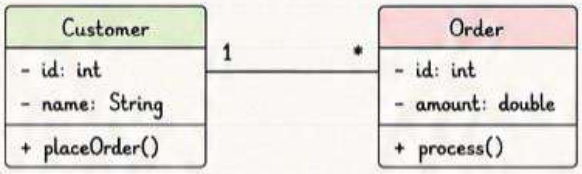
Visibility Modifiers

Symbol	Visibility	Meaning
+	Public	Accessible from anywhere
-	Private	Accessible only inside the class
#	Protected	Accessible in the same package and subclasses
~	Package / Default	Accessible within the same package

Relationships in UML

- **1. Association (Uses / Has-a)**

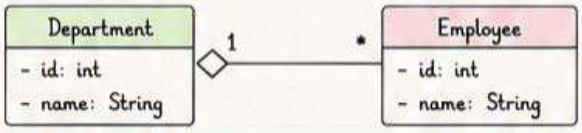
  A general relationship where one class uses another class.
  Example: Customer places Order.



1 → One
* → Many

- **2. Aggregation (Shared Has-a)**

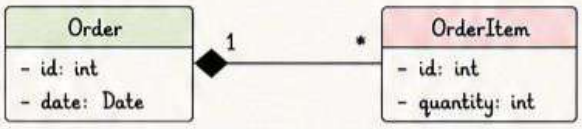
  A weak ownership. If the whole is destroyed, part can still exist.
  Example: Department has Employees.



Department is whole, Employee is part.

- **3. Composition (Strong Has-a)**

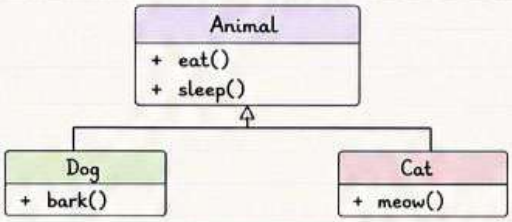
  A strong ownership. If the whole is destroyed, part cannot exist.
  Example: Order has OrderItems.



Order is whole, OrderItem is part (lifecycle dependent).

- **4. Inheritance (Is-a)**

  One class inherits properties and behavior from another.
  Example: Dog is an Animal.



Dog and Cat inherit from Animal.

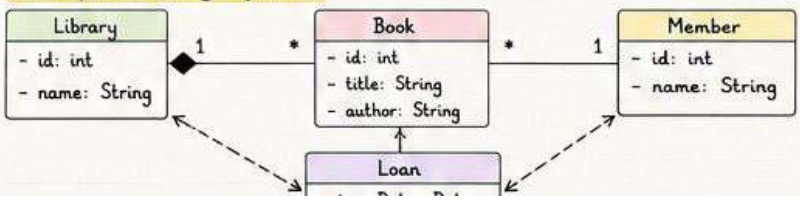
- **5. Dependency (Uses)**

  A temporary relationship. One class depends on another class temporarily.
  Example: Use of PaymentGateway inside PaymentService.



PaymentService temporarily uses PaymentGateway.

Example: Library System



Best Practices

- ✓ Keep classes small and focused.
- ✓ Avoid god classes.
- ✓ Define clear responsibilities.
- ✓ Use proper relationships.
- ✓ Prefer composition over inheritance.
- ✓ Follow SOLID principles.

Low-Level System Design (LLD) Handbook

4. Design Patterns (Creational)

Design patterns are reusable solutions to common problems in software design.

Why use Design Patterns?

- ✓ Solve recurring design problems.
- ✓ Promote code reusability and flexibility.
- ✓ Make code easier to maintain and extend.
- ✓ Provide proven, production-ready solutions.

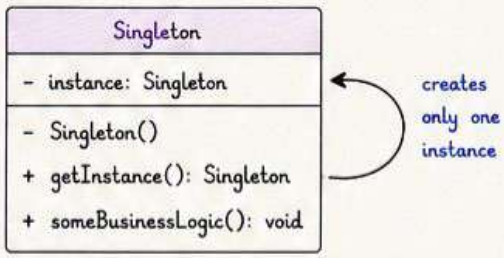
1. Singleton Pattern

Ensures a class has only one instance and provides a global point of access to it.

When to use:

- When exactly one instance is needed.
- For configuration, logger, cache, database connection.

Example: Logger, Configuration Manager, Cache



Key Benefit

Controls access to the sole instance.

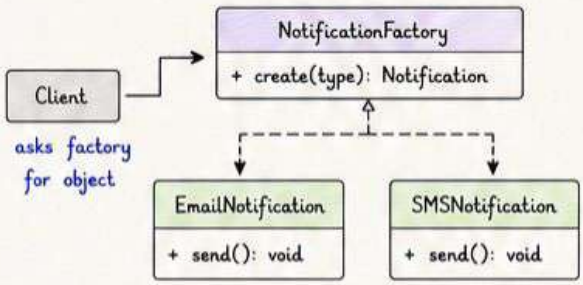
2. Factory Pattern

Defines an interface for creating an object, but lets subclasses decide which class to instantiate.

When to use:

- When a class can't anticipate the class of objects it must create.
- When you want to delegate instantiation logic.

Example: Notification Service, Payment Gateway



Key Benefit

Decouples client from concrete classes.

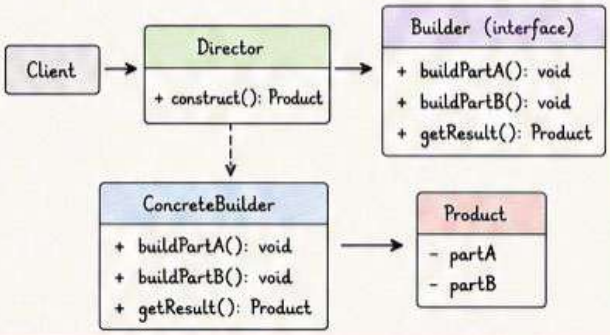
3. Builder Pattern

Separates the construction of a complex object from its representation so that the same construction process can create different representations.

When to use:

- When the object has many optional parameters.
- When step-by-step construction is needed.

Example: StringBuilder, Report Builder



Key Benefit

Creates complex objects step by step. Easy to extend.

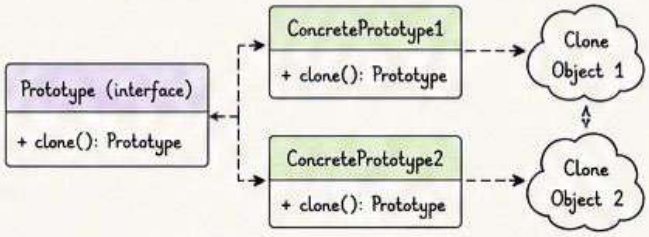
4. Prototype Pattern

Creates new objects by cloning an existing object, known as the prototype.

When to use:

- When object creation is expensive.
- When you want to avoid subclassing factories.

Example: Document, Shape, Game Entities



Key Benefit

Reduces cost of creating new objects.

Real-World Examples

Pattern	Real-World Use	Example
Singleton	Logging, Caching, Config	Logger.getInstance()
Factory	Payment, Notifications	PaymentFactory.create()
Builder	Reports, Documents	ReportBuilder.build()
Prototype	Cloning, Templates	Shape.clone()

Remember

Choose the right pattern based on the problem.

There is no one-size-fits-all solution!

Pro Tip

Focus on the problem you are solving, not the pattern itself.

Low-Level System Design (LLD) Handbook

5. Design Patterns (Structural)

Structural patterns are about organizing classes and objects to form larger structures while keeping them flexible and efficient.

Why Structural Patterns?

- ✓ Make systems easier to understand.
- ✓ Promote reuse of existing code.
- ✓ Hide implementation details.
- ✓ Improve flexibility and scalability.
- ✓ Help to compose objects and classes.

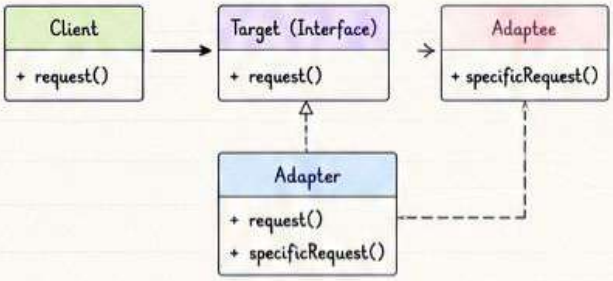
1. Adapter Pattern

Allows incompatible interfaces to work together.

When to use:

- When you want to use an existing class but its interface does not match.
- When integrating third-party libraries.

Example: Payment Gateway Integration



Key Benefit

Allows classes with incompatible interfaces to work together.

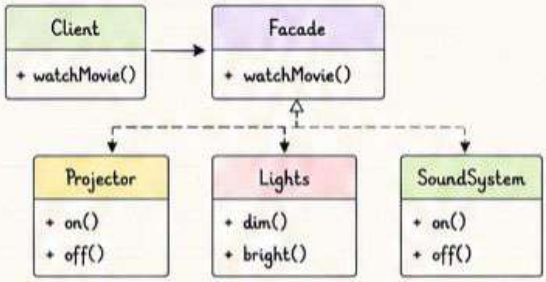
2. Facade Pattern

Provides a simplified interface to a complex subsystem.

When to use:

- When a system is complex and hard to use.
- When you want to provide a simple entry point.

Example: Home Theater System



Key Benefit

Hides complex subsystems behind a simple interface.

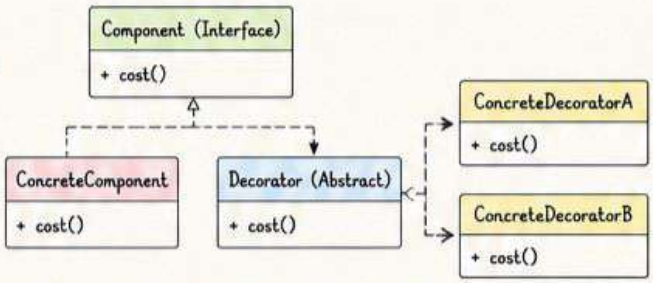
3. Decorator Pattern

Dynamically adds new responsibilities to an object.

When to use:

- When inheritance is not flexible.
- When you want to add behavior at runtime.

Example: Adding toppings to a Pizza



Key Benefit

Adds new behavior to objects dynamically.

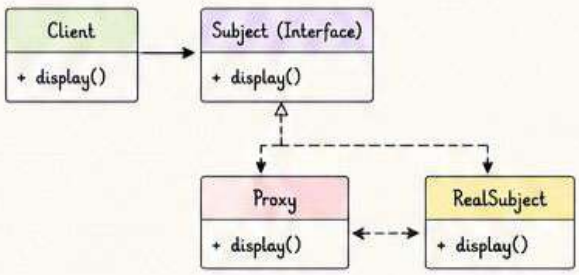
4. Proxy Pattern

Provides a surrogate or placeholder for another object.

When to use:

- When object creation is expensive.
- When you need access control.
- When you want lazy loading or caching.

Example: Virtual Proxy for Image Loading



Key Benefit

Controls access to the real object and can add extra functionalities.

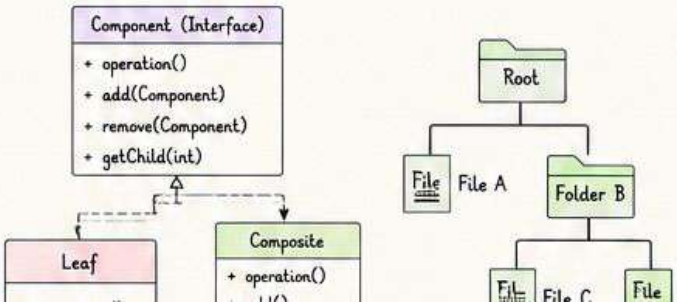
5. Composite Pattern

Composes objects into tree structures to represent part-whole hierarchies.

When to use:

- When you have a tree-like structure.
- When clients should treat individual objects and compositions uniformly.

Example: File System



Key Benefit

Treats individual objects and compositions uniformly.

Low-Level System Design (LLD) Handbook

6. Design Patterns (Behavioral)

Behavioral patterns focus on communication between objects and the assignment of responsibilities.

Why Behavioral Patterns?

- ✓ Improve communication between objects.
- ✓ Make systems more flexible and extensible.
- ✓ Encourage loose coupling.
- ✓ Help manage complex control flow.
- ✓ Make code easier to maintain and test.

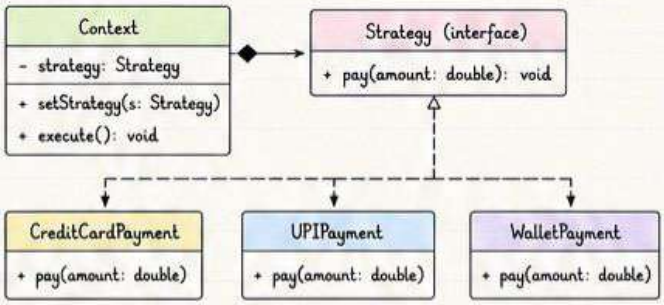
1. Strategy Pattern

Defines a family of algorithms, encapsulates each one, and makes them interchangeable.

When to use:

- When you have multiple algorithms.
- When you want to switch algorithms at runtime.

Example: Payment Gateway with multiple payment modes



Key Benefit

Algorithms can be changed without modifying the context.

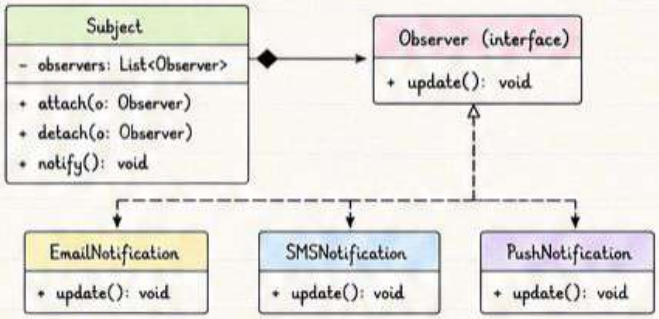
2. Observer Pattern

Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified automatically.

When to use:

- When a change in one object needs to update multiple objects.
- When you want loose coupling.

Example: Notification System



Key Benefit

New observers can be added without changing the subject.

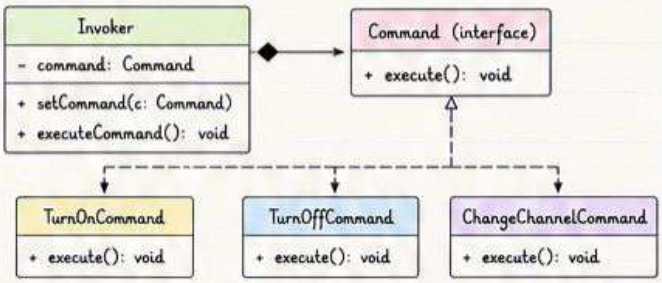
3. Command Pattern

Encapsulates a request as an object, thereby letting you parameterize clients with different requests.

When to use:

- When you want to parameterize objects with operations.
- When you want to queue, log or undo operations.

Example: Remote Control



Key Benefit

Supports undo, logging and queuing of commands.

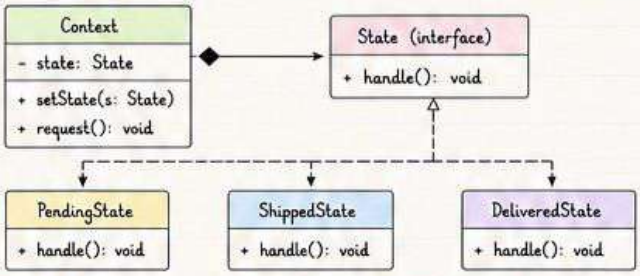
4. State Pattern

Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

When to use:

- When an object has different states.
- When state-dependent behavior is complex.

Example: Order State Management



Key Benefit

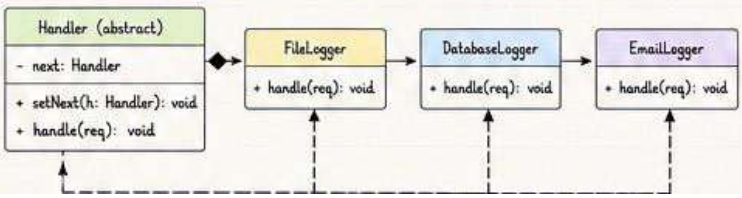
Eliminates large conditional statements.

5. Chain of Responsibility Pattern

Passes a request along a chain of handlers. Each handler decides either to process the request or to pass it to the next handler in the chain.

When to use:

- When more than one object can handle a request.
- When you want to decouple sender and receiver.



Key Benefit

Decouples sender and receiver. Easy to extend the chain.

Low-Level System Design (LLD) Handbook

7. Designing Parking Lot System

Let's design a Parking Lot System using OOP principles and design patterns.

Requirements

- Parking lot has multiple floors.
- Each floor has multiple slots.
- Supports different vehicle types.
- Vehicle can park and unpark.
- Generate ticket while parking.
- Calculate parking charges.
- Multiple payment modes.

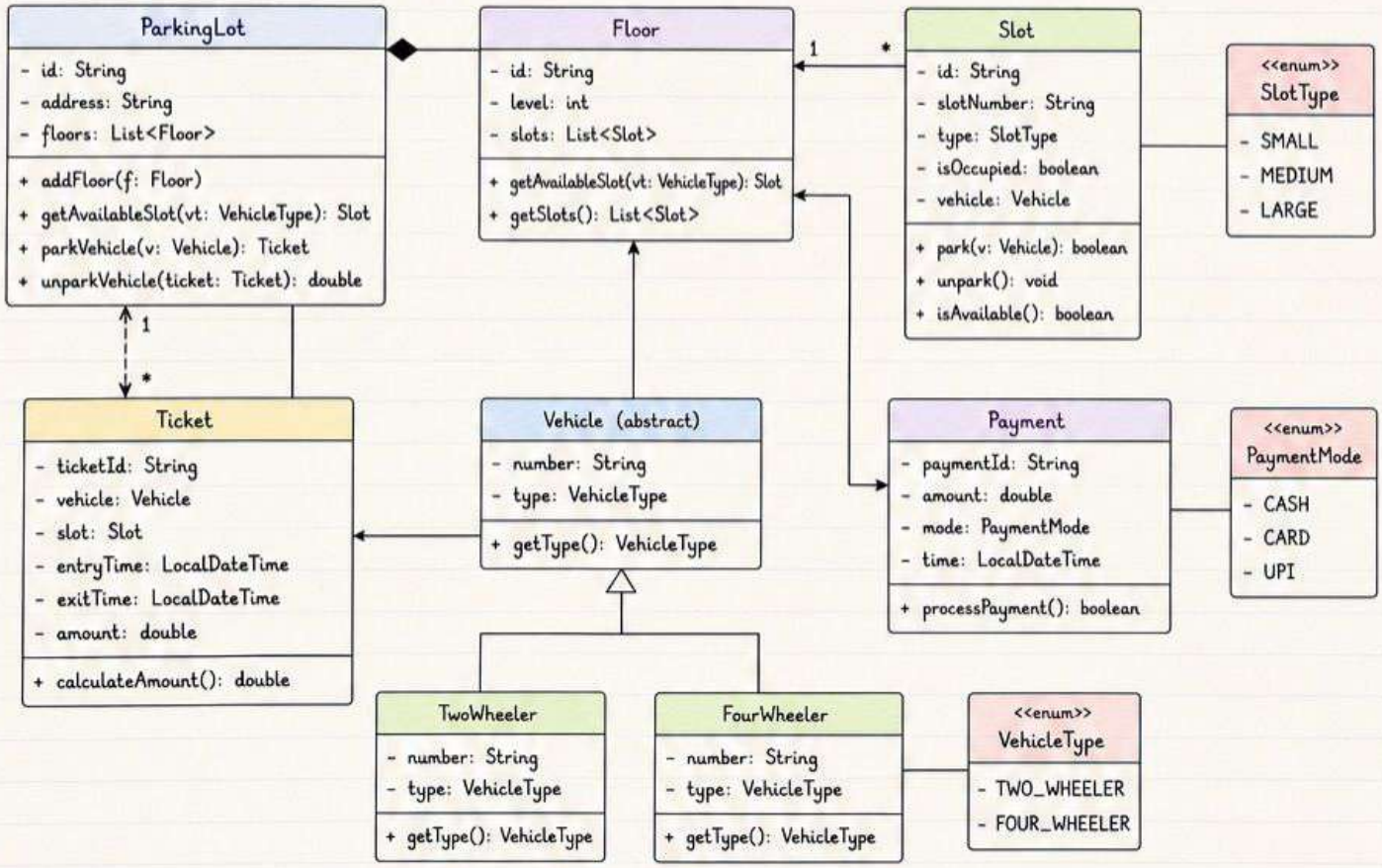
. High Level Approach

1. Identify entities (classes)
2. Define their attributes and methods
3. Define relationships
4. Apply OOP principles
5. Consider extensibility
6. Handle edge cases

Key Considerations

- Two-wheeler takes 1 slot
- Four-wheeler takes 1 slot
- Check for available slot
- Ticket has entry time
- Charges depend on duration and vehicle type

1. Class Diagram



2. Key Class Responsibilities

- ParkingLot : Manages floors and parking operations.
- Floor : Manages slots on a particular floor.
- Slot : Knows if it is available or occupied.
- Vehicle : Base class for different vehicle types.
- Ticket : Stores parking details and calculates charges.
- Payment : Handles payment processing.

3. Sample Flow (Parking a Vehicle)

1. Vehicle arrives at parking lot.
2. System finds an available slot based on vehicle type.
3. Vehicle is parked in the slot.
4. Ticket is generated with entry time.
5. On exit, duration is calculated and amount is generated.
6. Payment is processed and vehicle is unparked.

Interview Tip

Common Edge Cases

Low-Level System Design (LLD) Handbook

8. Designing BookMyShow System

Let's design a simplified BookMyShow system using OOP principles and design patterns.

Requirements

- Users can browse movies.
- Check show timings and availability.
- Select seats and book tickets.
- Multiple payments modes.
- Send booking confirmation.
- Handle cancellations and refunds.

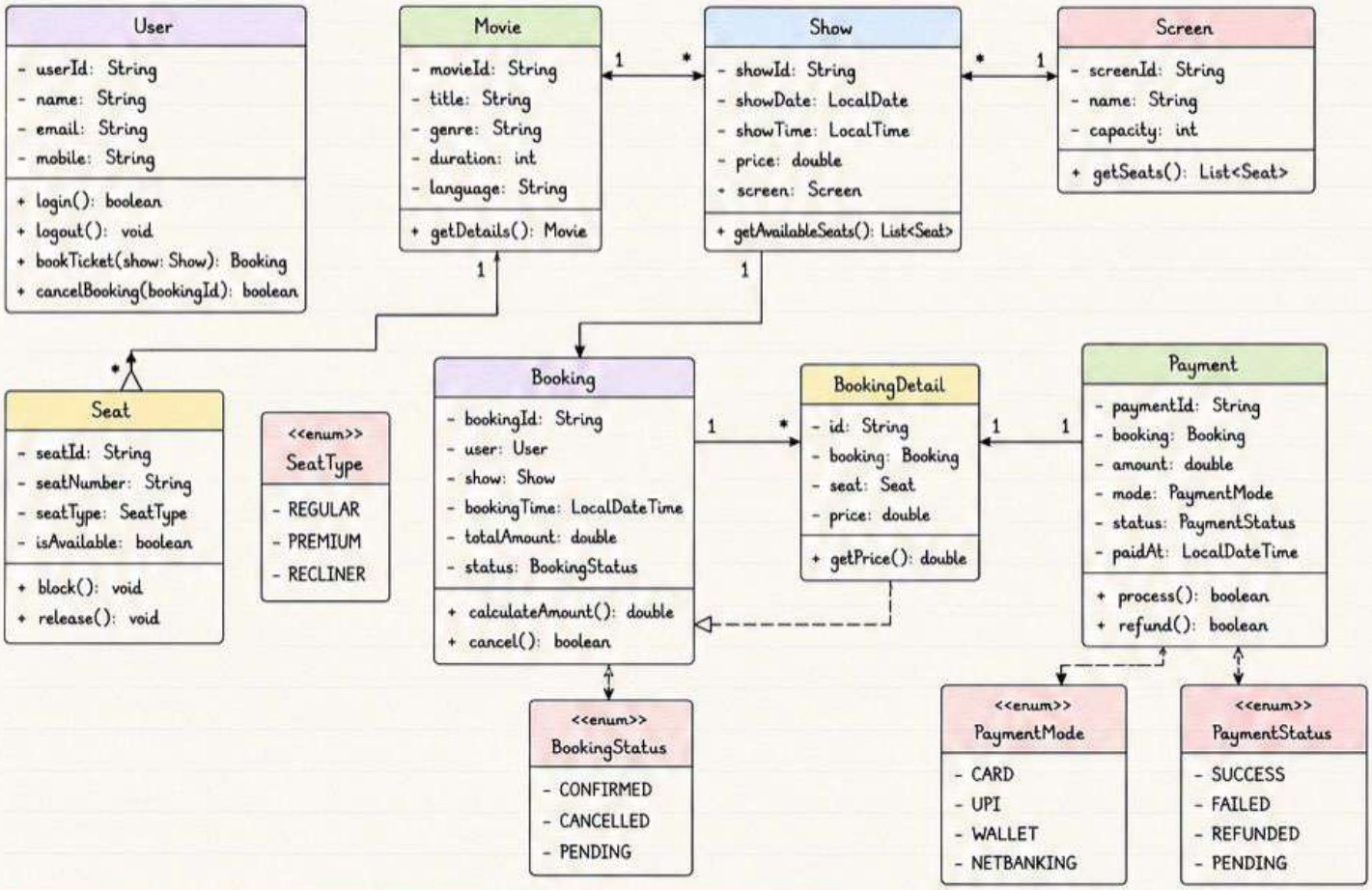
High Level Approach

1. Identify entities (classes)
2. Define attributes and methods
3. Define relationships
4. Apply OOP principles
5. Consider design patterns
6. Handle edge cases

Key Considerations

- Seat availability in real-time
- Multiple shows for a movie
- Different seat categories
- Payment must be secure
- Handle high traffic during peak times
- Handle cancellations

1. Class Diagram



2. Key Class Responsibilities

- User : Manages user actions like booking, cancellation.
- Movie : Stores movie information.
- Show : Represents a particular show of a movie.
- Screen : Holds seat layout and configuration.
- Seat : Knows its availability and type.
- Booking : Manages booking lifecycle.
- BookingDetail : Stores seats and price info for a booking.
- Payment : Handles payment and refunds.

3. Design Patterns Used

- Factory Pattern : For creating Payment objects based on PaymentMode.
- Strategy Pattern : For different payment methods.
- Observer Pattern : To send notifications after booking or cancellation.
- Singleton Pattern : For configuration management (e.g., Seat layout cache).

Sample Flow (Book Tickets)

Edge Cases

Interview Tip

Low-Level System Design (LLD) Handbook

9. SOLID Principles (For Better Design)

SOLID principles help us write clean, maintainable and scalable code.

S Single Responsibility Principle (SRP)
A class should have only one reason to change, i.e., one responsibility.

O Open/Closed Principle (OCP)
A class should be open for extension but closed for modification.

L Liskov Substitution Principle (LSP)
Objects of a superclass should be replaceable with objects of its subclasses.

I Interface Segregation Principle (ISP)
Clients should not be forced to depend on interfaces they do not use.

D Dependency Inversion Principle (DIP)
High-level modules should not depend on low-level modules. Both should depend on abstractions.

Why SOLID?

- ✓ Easy to understand and modify.
- ✓ Reduces bugs and code duplication.
- ✓ Improves code reusability.
- ✓ Makes the system flexible and scalable.
- ✓ Industry standard for good design.

Example: SRP (Bad vs Good)

Bad (Multiple Responsibilities)

```
class UserManager {
   void createUser() { ... }
   void sendEmail() { ... }
   void saveToDB() { ... }
}
```

✗ Hard to change and test

Good (Single Responsibility)

```
class UserService {
   void createUser() { ... }
}
class EmailService {
   void sendEmail() { ... }
}
class UserRepository {
   void saveToDB() { ... }
}
```

✓ Easy to change and test

10. Common LLD Best Practices

- ✓ Use meaningful class and method names.
- ✓ Keep classes small and focused.
- ✓ Prefer composition over inheritance.
- ✓ Use interfaces to achieve abstraction.
- ✓ Follow DRY (Don't Repeat Yourself).
- ✓ Handle exceptions properly.
- ✓ Write unit tests for important components.
- ✓ Keep the code simple and YAGNI (You Aren't Gonna Need It).

Composition over Inheritance (Example)

Inheritance (Tight Coupling)

```
class Vehicle {
   void move() { ... }
}

class Car extends Vehicle {
   void move() { ... }
}
```

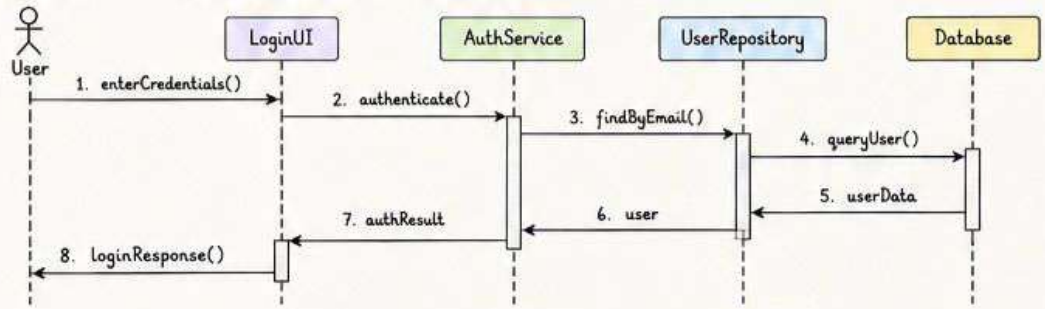
✗ Rigid and hard to extend

Composition (Loose Coupling)

```
interface Movable {
   void move();
}
class Car {
   private Movable movable;
   Car(Movable m) {
      this.movable = m;
   }
   void move() {
      movable.move();
   }
}
```

✓ Flexible and easy to extend

11. Sequence Diagram Example: User Login



What it shows?

- Flow of messages between objects.
- Helps to understand the interaction in a system.

12. Useful Tips

- Start with a clear understanding of requirements.
- Identify key entities, attributes and relationships.
- Design classes and interfaces before writing code.
- Review and refactor regularly.
- Keep learning new patterns and best practices.

LLD Mindset

- ✓ Think like a designer.
- ✓ Design for change.
- ✓ Keep it modular.
- ✓ Keep it simple.

Pro Tip

Good design is not about writing more code, it's about organizing code in the right way!

Low-Level System Design (LLD) Handbook

13. LLD vs HLD

	LLD	HLD
Focus	Detailed design of individual components	Overall system architecture and structure
Audience	Developers	Architects, Tech Leads, Stakeholders
Scope	Narrow (Class level, module level)	Wide (System level, service level)
Abstraction	Low level (Detailed)	High level (Abstract)
Artifacts	Class diagrams, Sequence diagrams, Code logic	System architecture diagram, Component diagram
Purpose	How the system will be implemented	What the system will do and how components interact

Key Takeaway

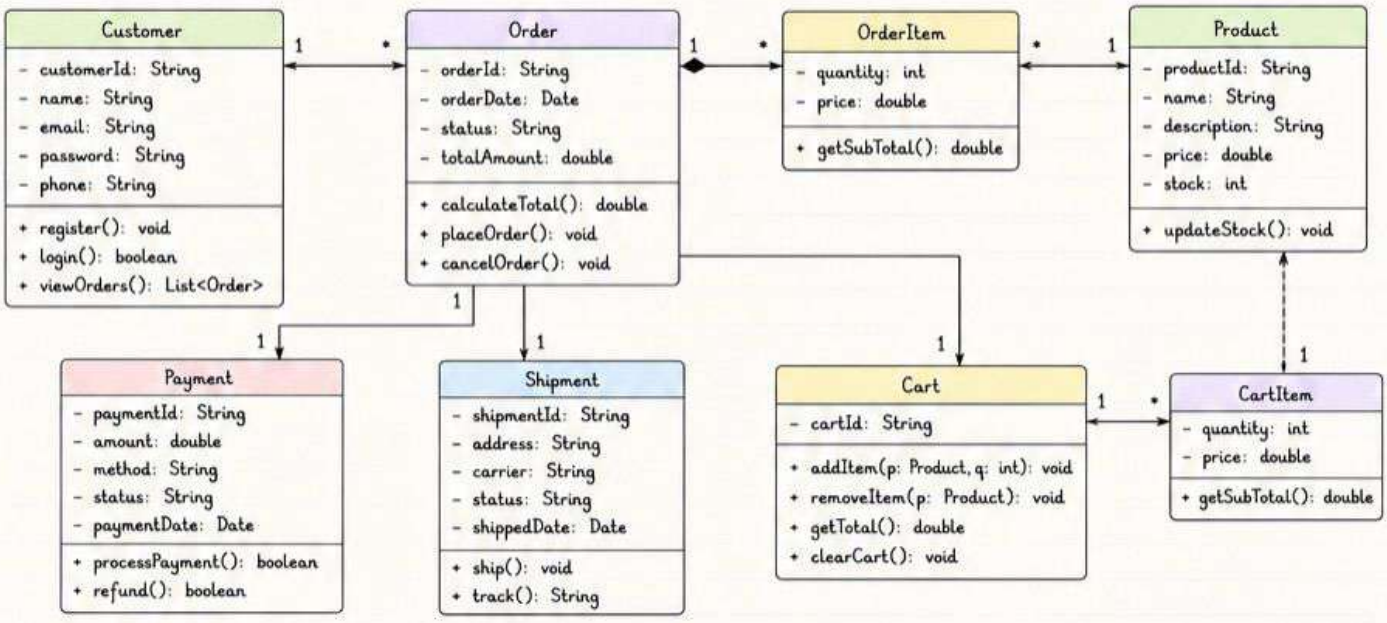
- HLD is the blueprint.
- LLD is the detailed plan.
- Both are important for building a strong system.



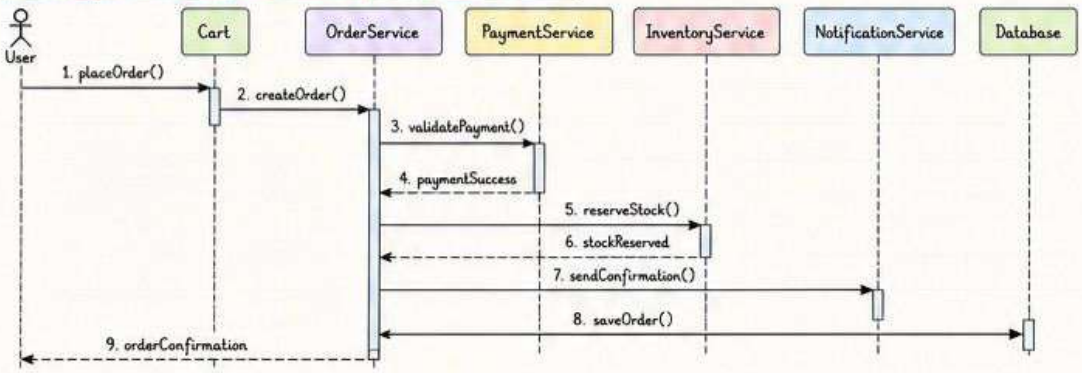
When to do LLD?

- After HLD is approved.
- Before actual coding.
- When a module/feature is well understood.
- When system needs to be maintained by multiple developers.

14. Class Diagram Example: Online Shopping System



15. Sequence Diagram Example: Place Order



What it shows?

- Sequence of interactions between objects.
- Helps to understand the flow of a use case.
- Useful for identifying responsibilities and dependencies.

16. Design Checklist

- ✓ Have I identified entities and their responsibilities?
- ✓ Are relationships and associations clear?
- ✓ Are interfaces and access modifiers correct?

17. Final Thoughts

"Good LLD is not about writing more code, it's about writing better code."

- ✓ Keep learning.

Interview Tip

Always think out loud.
Ask clarifying questions.
Start with simple design and iterate.